INFOSCI201 Physical Computing
Nemuulen Togtbaatar

Documentation: Arduino desk lamp with sensors and a timer

1. Project overview

This project is an Arduino-based interactive desk lamp that combines ambient sensing, manual controls, and a Pomodoro-style timer. It was developed iteratively, with several design changes and failures along the way to reach a stable final version.

2. Project features

- In “SENSOR” mode, the lamp uses ambient light to set brightness and temperature to set color. As the ambient light level increases, the LED becomes brighter, and as the surrounding temperature rises, the lamp shifts toward a cooler, bluish tone. This behavior is intended to maintain comfortable, visually balanced lighting that automatically responds to changes in the environment.
- “MANUAL” mode with two potentiometers for independent brightness and color control.
- Pomodoro-style timer with 25/5 minute work–break cycles.
- Button interactions: single tap, double tap, and long press for different timer/mode actions.
    - Left button:
        - Single press: Toggles between sensor/manual mode.
        - Double press: Resets timer
    - Right button
        - Single short press: If no timer is running, it starts a 5-minute countdown. If a timer is running, it pauses the countdown. If a timer is paused, it resumes from the remaining time.
        - Double short press: Adds 5 minutes.
        - Long press: Starts continuous Pomodoro mode.
- The buzzer gives audio feedback when buttons are pressed and upon timer completion.
- The LCD screen displays mode, timer state, and sensor readings.

3. System design

3.1 Hardware components

- Arduino Uno.
- RGB LED (R - D9, G - D10, B - D11).
- Ambient light sensor on A2.
- Temperature sensor on A3.
- Two potentiometers:
  - A0: overall brightness.

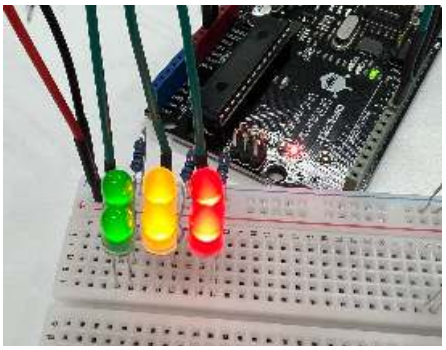
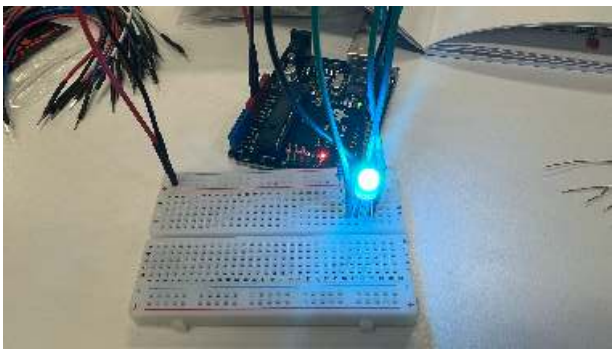
  - A1: color temperature/hue.
- Two push buttons:
  - D12: mode button.
  - D13: timer button.
- Buzzer on D4.
- DFRobot RGB LCD1602 (SDA = A4, SCL = A5).
- Breadboard and jumpers

3.2 Software and tools

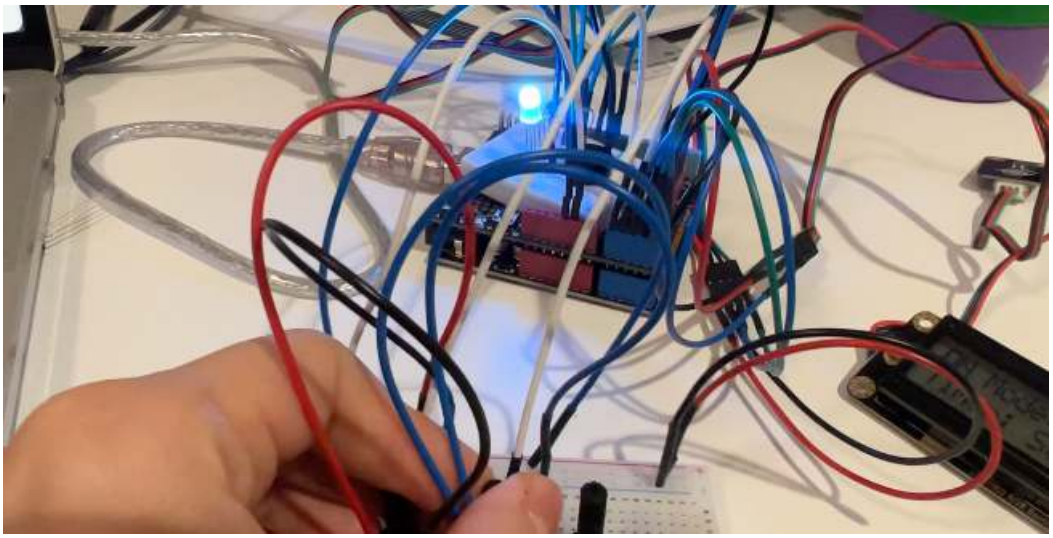
- Arduino IDE

4. Development process and iterations

1. I started with whether the 5mm LED in parallel is brighter than the RGB LED light, and it turned out that the RGB LED light can be much brighter.

2. Tested RGB LED color and brightness control with simple test code.

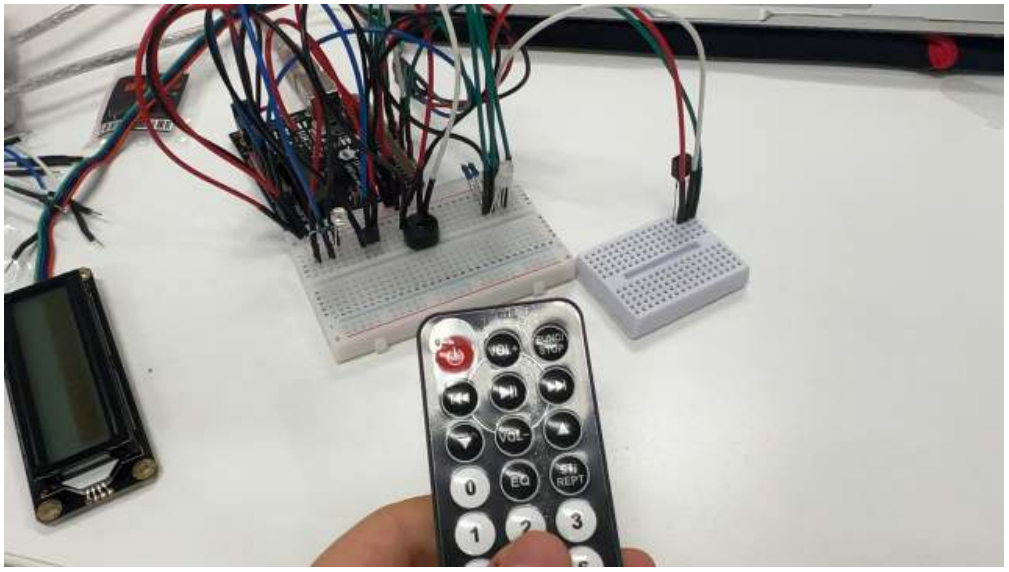


3. Attempted the IR remote control and successfully got the signals for each button.

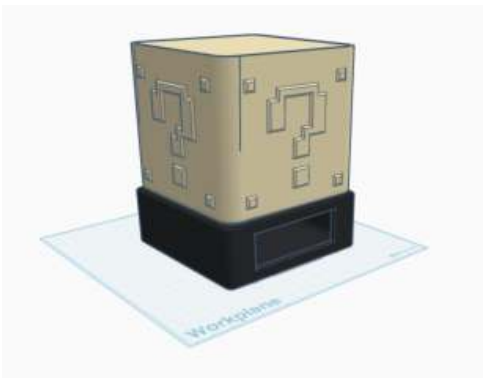
To get controller command codes, I followed this tutorial:
https://www.instructables.com/Arduino-Infrared-Remote-tutorial/https://youtu.be/q97VE3oEwIc

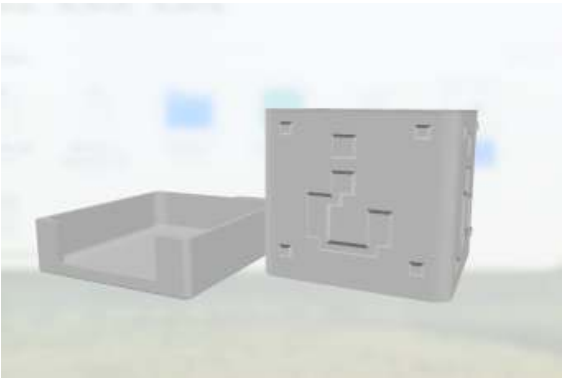
```
// NEC remote with address 0xBF00
const uint16_t IR_ADDR = 0xBF00;
```

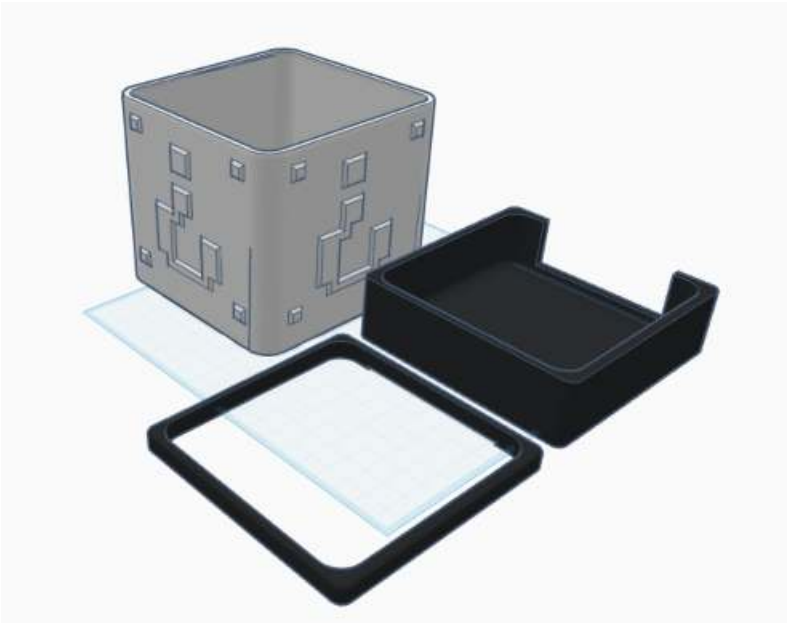
```
// Top row
const uint8_t CMD_PWR        = 0x00  // pwr (turn on the whole system)
const uint8_t CMD_VOL_PLUS   = 0x01;  // vol+ (add 5min, 5 minute timer)
const uint8_t CMD_FUNC       = 0x02;  // (change mode auto/manual)
const uint8_t CMD_LEFT       = 0x04;  // left (hue colder)
const uint8_t CMD_PLAYPAUSE  = 0x05;  // resume/pause
const uint8_t CMD_RIGHT      = 0x06;  // right (hue warmer)

// Middle row
const uint8_t CMD_UP         = 0x0A;  // up (brightness up)
const uint8_t CMD_VOL_MINUS  = 0x09;  // vol- (subtract 3 minutes)
const uint8_t CMD_DOWN       = 0x08;  // down (brightness down)
const uint8_t CMD_0_TOP      = 0x0C  // 0 button
const uint8_t CMD_EQ         = 0x0D;  // eq
const uint8_t CMD_ST         = 0x0E  // (25min timer + 5min break loop)

const uint8_t CMD_NUM_1      = 0x10;  // 1
const uint8_t CMD_NUM_2      = 0x11  // 2
const uint8_t CMD_NUM_3      = 0x12  // 3
const uint8_t CMD_NUM_4      = 0x14  // 4
const uint8_t CMD_NUM_5      = 0x15;  // 5
const uint8_t CMD_NUM_6      = 0x16;  // 6
const uint8_t CMD_NUM_7      = 0x18;  // 7
const uint8_t CMD_NUM_8      = 0x19;  // 8
const uint8_t CMD_NUM_9      = 0x1A;  // 9
```



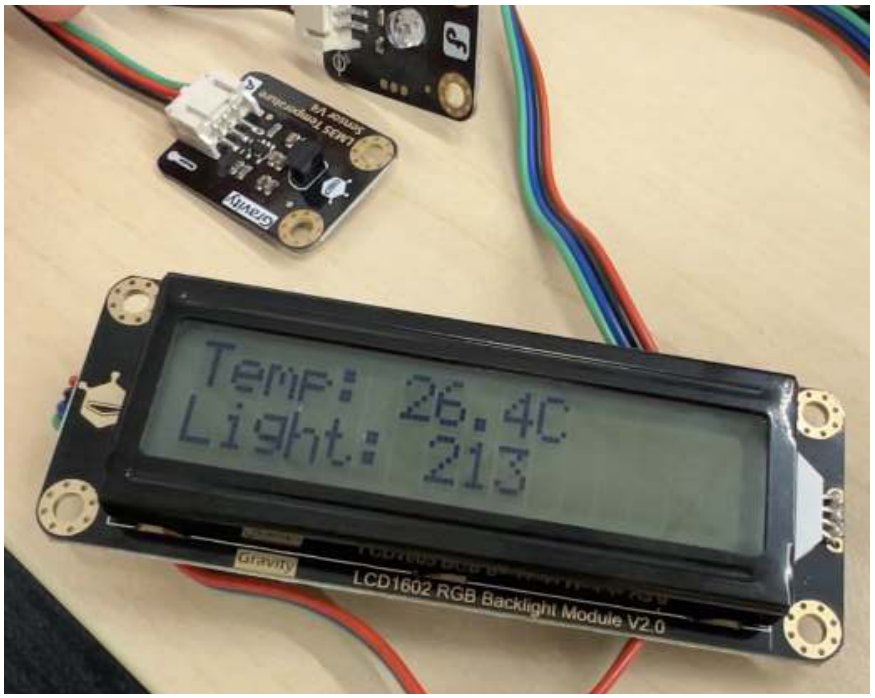
4. Designed the outer shell of the lamp on TinkerCAD.





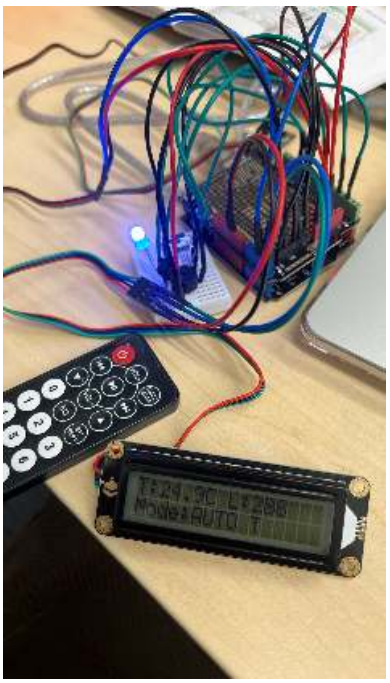


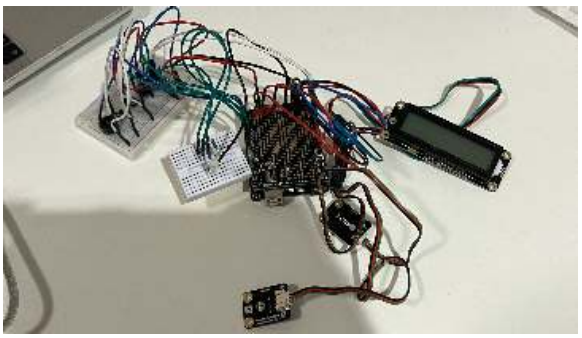
5. Added ambient light and temperature sensors to create an adaptive mode.
6. Tested the temperature and light sensor function by printing them on the screen.



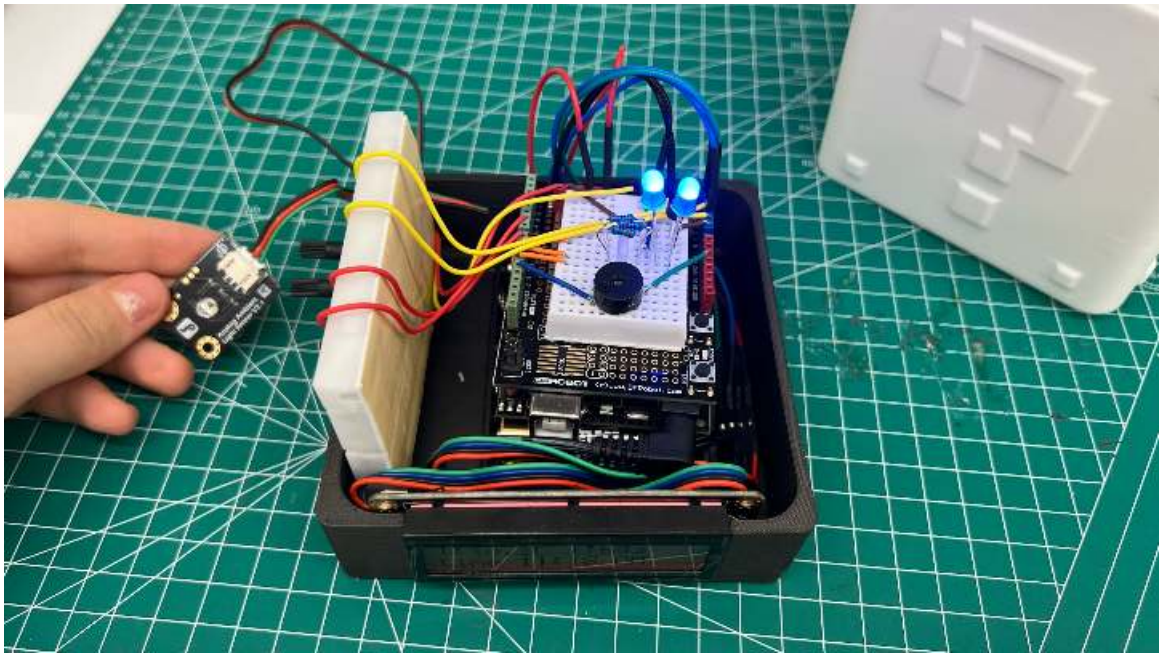
7. However, due to an unknown issue (either a battery or a connection issue), the controller stopped working.
8. After multiple attempts and various library tests and using another controller, the IR sensor was removed, and the design reverted to physical buttons and potentiometers.
9. Implemented a basic single timer first, then extended it to a Pomodoro cycle.
10. Added the button logic and learned ways to use different signals (single tap, double tap, long press) for efficiency.
11. Cleaned up code and comments.

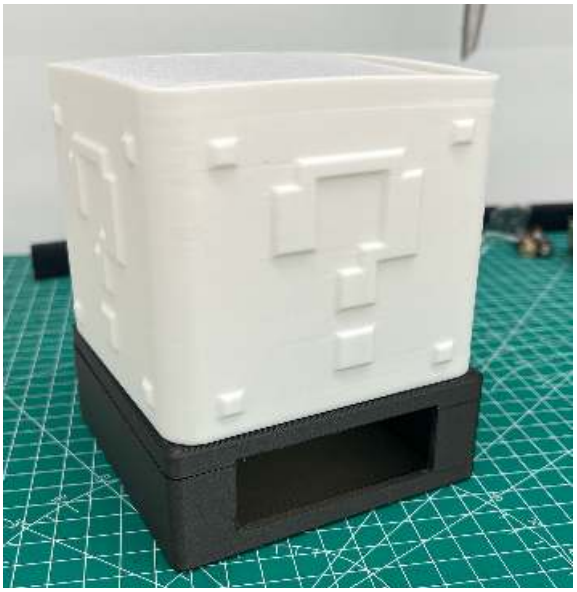
Iterations:

Final look:





Videos are included in the folder.